

Overall Lab Schedule

Experiments	Content	Week
1	Aircraft War System Analysis and Singleton Pattern	Week 11 Monday: 18:45-22:05 PM Room T2510 
2	Factory Pattern	Week 12 Thursday: 14:00-17:20 PM Room T2510 
 3	Strategy Pattern	Week 14 Tuesday: 14:00-17:20 PM Room T2510 
4	Audio Control and Multithreading	Week 16 Monday: 14:00-17:20 PM Room T2510 



Experiment 3: Strategy Pattern

Objective

- Understand the motivation and design intent of the Strategy Pattern
- Master the structure of the Strategy Pattern and be able to draw its UML class diagram
- Refactor existing code and implement the Strategy Pattern in practice



Experiment Tasks

1. Design Pattern Application

Identify scenarios suitable for the **Strategy Pattern and implement it**

- Draw the Strategy Pattern UML class diagram for the aircraft war scenario
- Refactor the code to apply the pattern for implementing different bullet trajectories and firepower supply effects

2. Iterative Development Goal

Two types of firepower supply take effect

- **Firepower Supply** : Changes the hero's bullet trajectory to spread shot
- **Super Firepower Supply**  **(new requirement)**: Changes the hero's bullet trajectory to circular shot

Experiment Steps: Strategy Pattern (1)

➤ Why do we need the Strategy pattern?

Scenario Analysis

Requirement:

In the Aircraft War, different types of aircraft have distinct bullet trajectories, and the hero aircraft switches its trajectory when a firepower supply takes effect.

Experiment Steps: Strategy Pattern (2)

➤ Why do we need the Strategy pattern?

A diagram consisting of two concentric circles. The inner circle is blue and contains the text "Scenario Analysis" in white. The outer circle is a lighter shade of blue.

Scenario
Analysis

Aircraft		Bullet Trajectory
Mob Enemy		No shooting
Elite Enemy		straight shot
Boss Enemy		circular shot
Hero	No Supply	straight shot
	Firepower Supply	spread shot
	Super Firepower Supply	circular shot

Experiment Steps: Strategy Pattern (3)

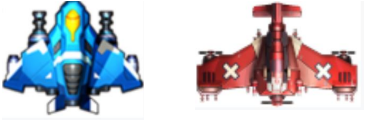


Let's think about following questions

(1) How do we currently implement the bullet trajectory of aircraft?

```
@Override
public List<BaseBullet> shoot() {
    List<BaseBullet> res = new LinkedList<>();
    int x = this.getLocationX();
    int y = this.getLocationY() + direction*2;
    int speedX = 0;
    int speedY = this.getSpeedY() + direction*5;
    BaseBullet bullet;
    for(int i=0; i<shootNum; i++){
        bullet = new HeroBullet( locationX: x + (i*2 - shootNum + 1)*10,
                                y, speedX, speedY, power);
        res.add(bullet);
    }
    return res;
}
```

Straight Shot



```
@Override
public List<BaseBullet> shoot() {
    List<BaseBullet> res = new LinkedList<>();

    int x = this.getLocationX();
    int y = this.getLocationY();

    int bulletSpeed = 10;
    int bulletRadius = 100;
    int angleIncrement = 360 / shootNum;

    BaseBullet bullet;
    for (int i = 0; i < shootNum; i++) {
        int angle = i * angleIncrement;
        int bulletX = (int) (x + bulletRadius * Math.cos(Math.toRadians(angle)));
        int bulletY = (int) (y + bulletRadius * Math.sin(Math.toRadians(angle)));
        int bulletSpeedX = (int) (bulletSpeed * Math.cos(Math.toRadians(angle)));
        int bulletSpeedY = (int) (bulletSpeed * Math.sin(Math.toRadians(angle)));
        bullet = new EnemyBullet(bulletX, bulletY, bulletSpeedX, bulletSpeedY, power);
        res.add(bullet);
    }
    return res;
}
```

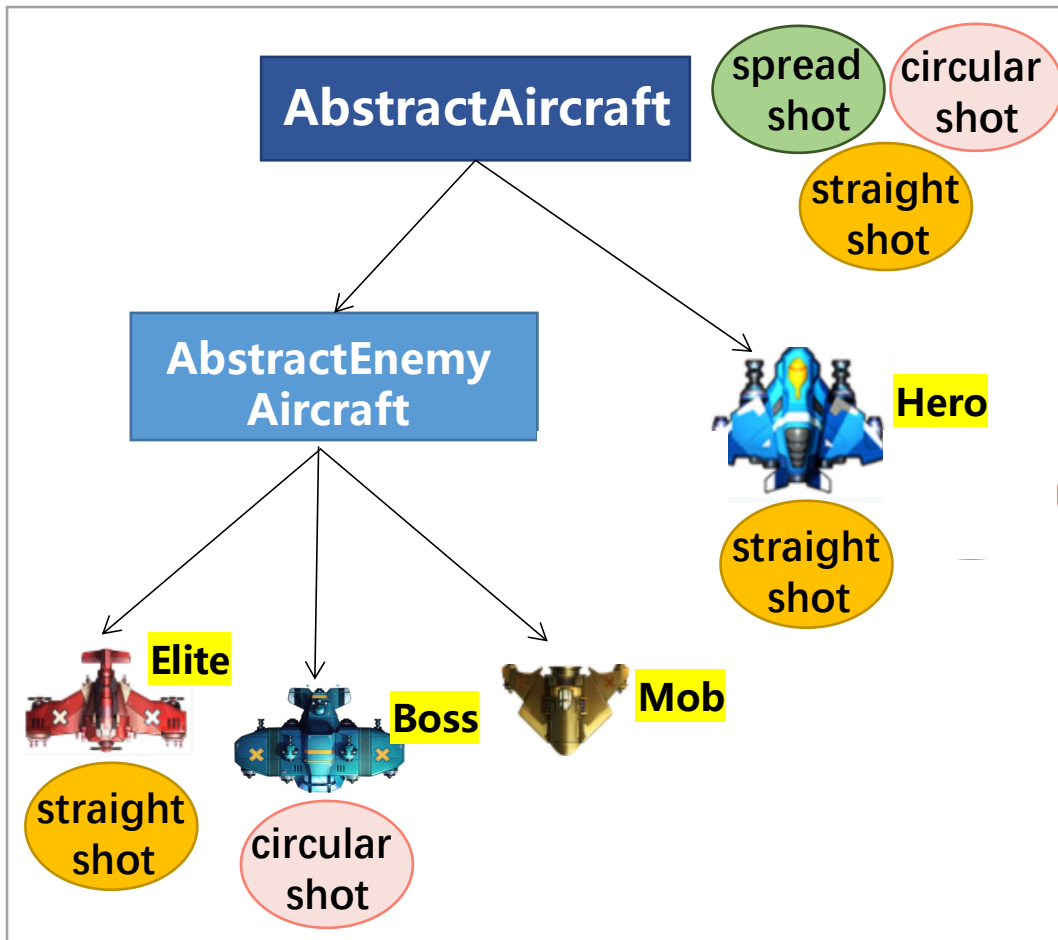
Circular Shot



Experiment Steps: Strategy Pattern (4)



(2) How to make the firepower supply take effect?



Method 01

Implement the spread shot and circular shot in Hero class.

Too much duplicate code;
Code is hard to maintain;
Prone to bugs.

Violate
Open-Closed
Principle



Method 02

Put the code for the three trajectories into the parent class for inheritance.

Multiple selections;
Complex logic;
Poor code readability.

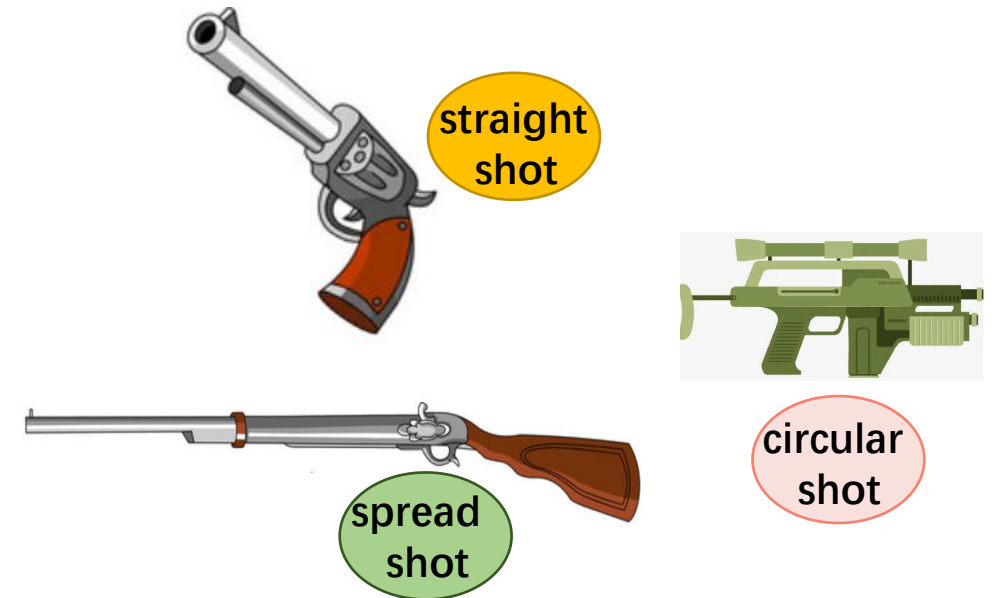
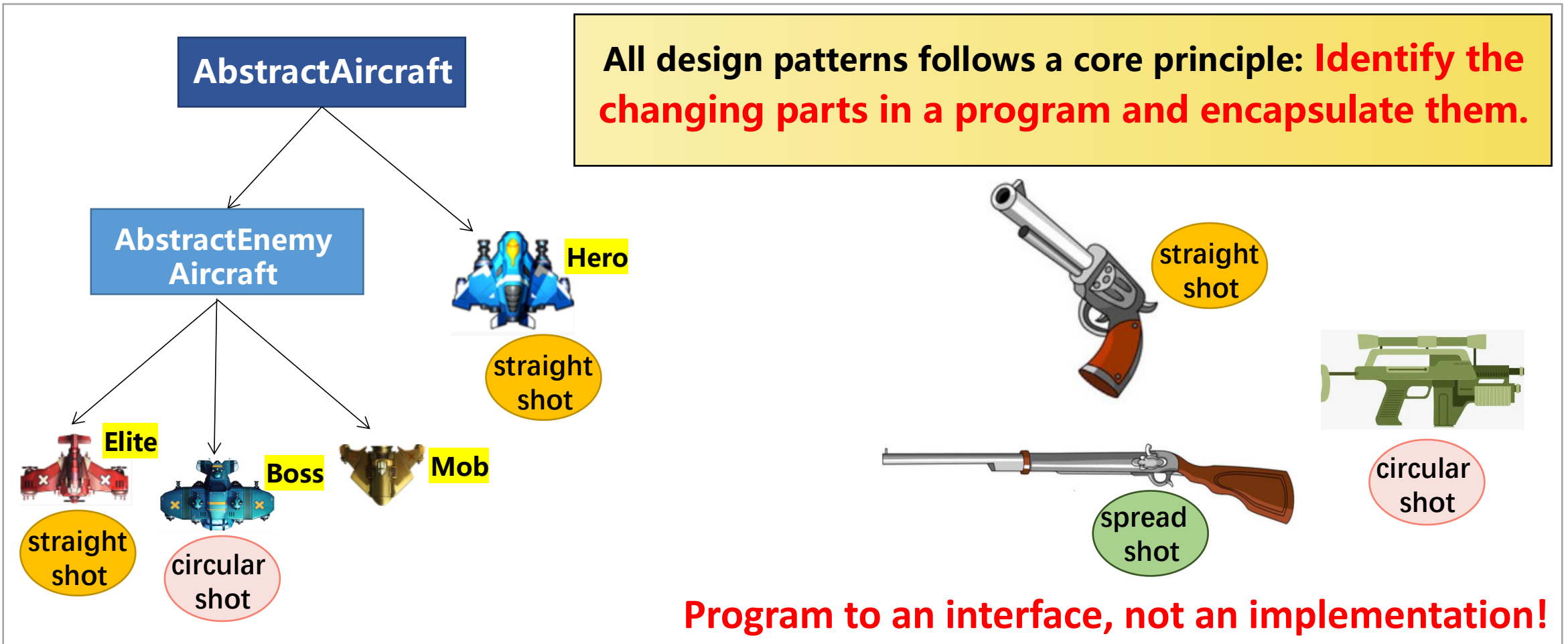
Violate
Composition
Reuse Principle



Experiment Steps: Strategy Pattern (5)



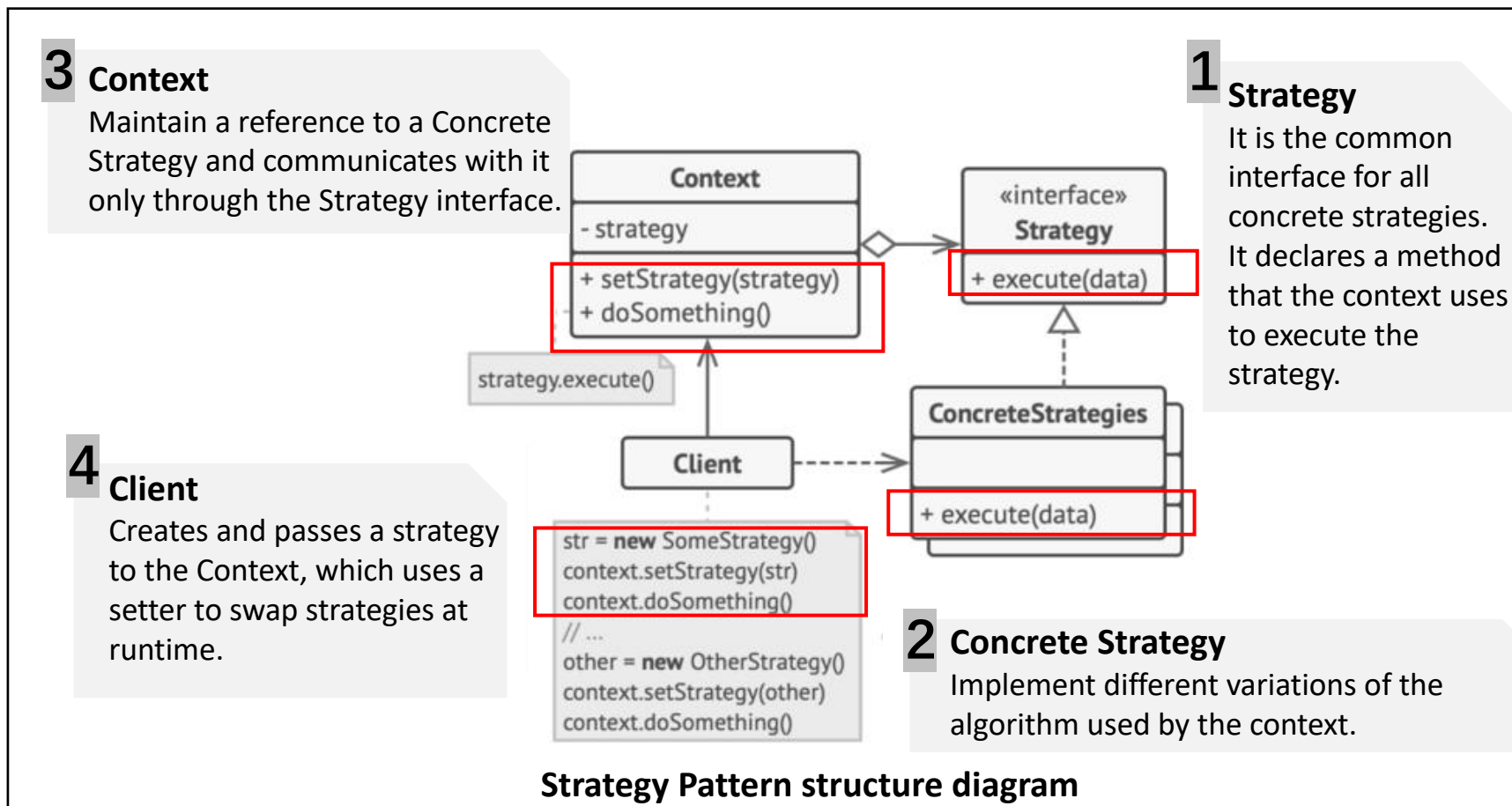
How can we use object composition instead of inheritance to achieve code reuse?



Program to an interface, not an implementation!

Experiment Steps: Strategy Pattern (6)

Strategy Pattern allows you to define a family of algorithms, encapsulate each one into a separate class, and make their objects interchangeable.



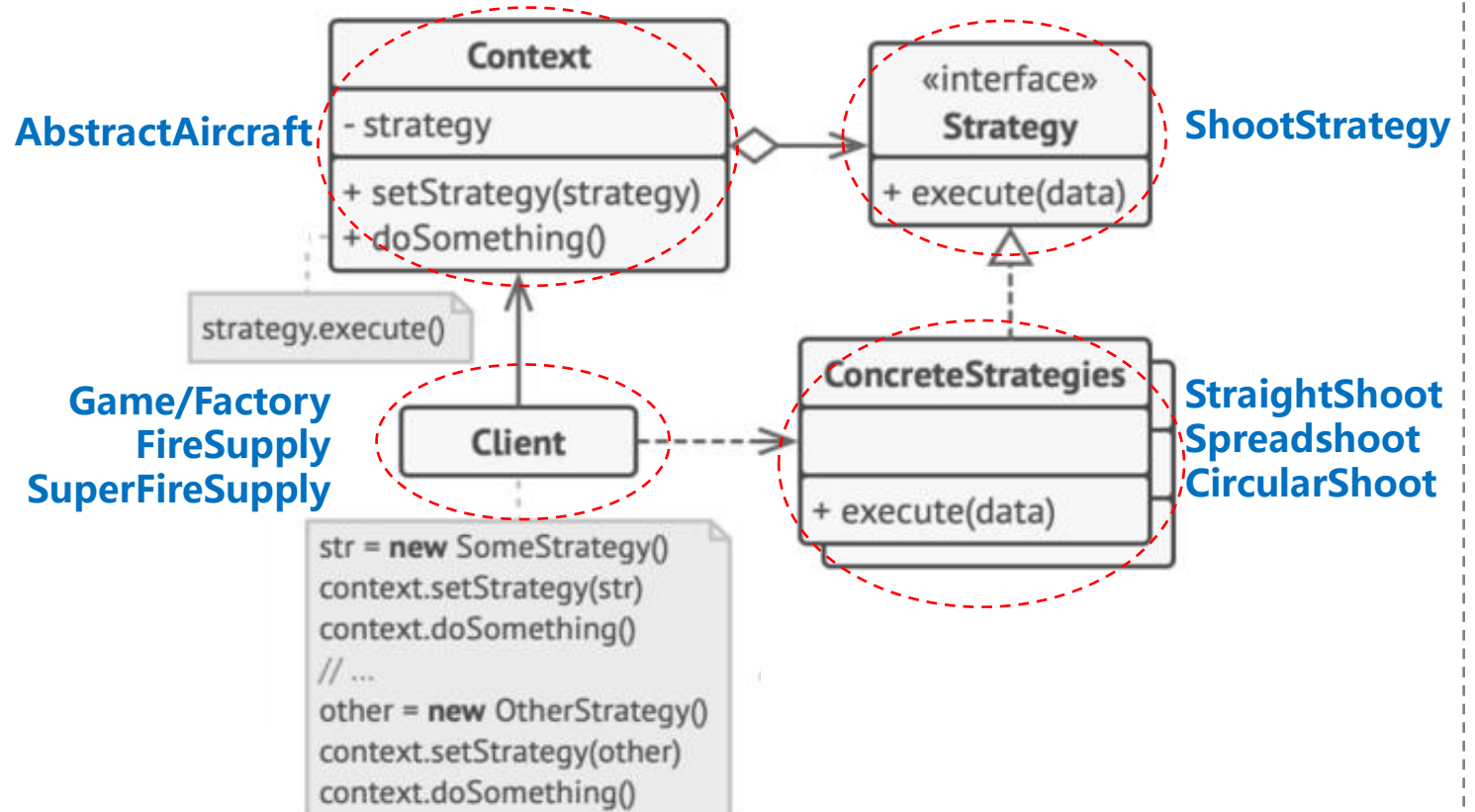
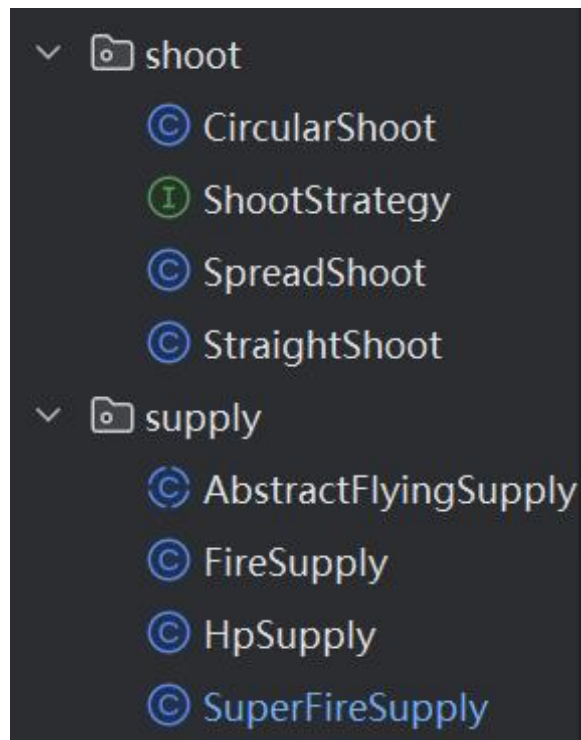
Experiment Steps: Strategy Pattern (7)

➤ Tasks to be completed:

- **Draw** the Strategy Pattern **UML class diagram** for aircrafts and different bullet trajectories in this game. The client side does not need to be drawn.
- **Refactor** the code based on the **Strategy Pattern** you designed to implement three types **of bullet trajectories**: straight shot, spread shot and circular shot. Ensure that the hero aircraft and the three types of enemy aircraft use the Strategy Pattern to implement bullet trajectory.

Experiment Steps: Strategy Pattern (8)

- How should we design if we adopt the Strategy Pattern to implement different bullet trajectories?



Experiment Steps: Strategy Pattern Example (1)

Suppose we want to implement a **calculator** that supports four operations: **addition**, **subtraction**, **multiplication**, and **division**. How can we apply the Strategy Pattern to achieve this?

How should we implement this using the Strategy Pattern?

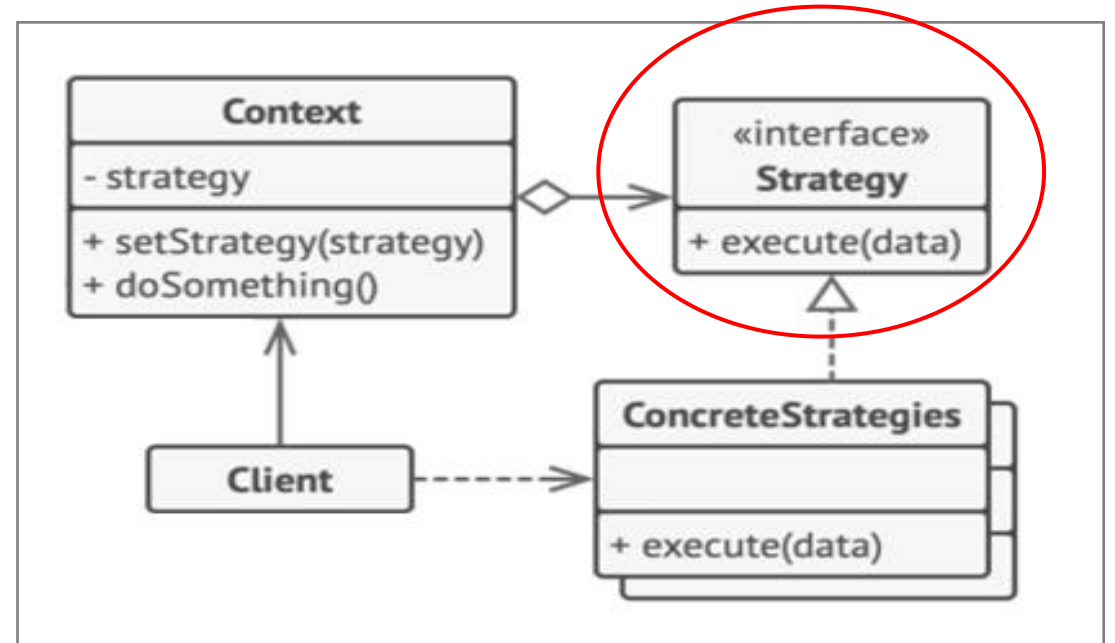


Experiment Steps: Strategy Pattern Example (2)

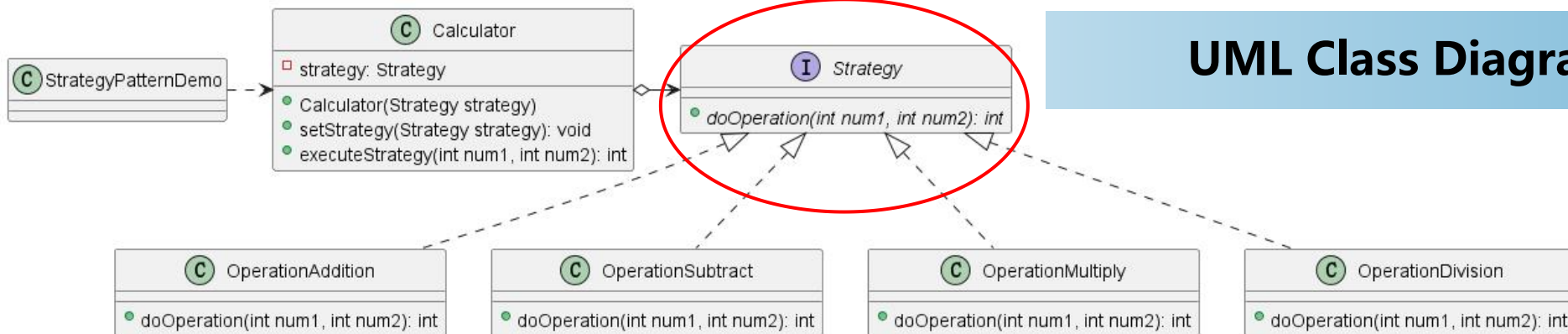
Code Implementation

- ① Create an interface to act as the abstract Strategy role

```
public interface Strategy {  
    int doOperation(int num1, int num2);  
}
```



UML Class Diagram Design



Experiment Steps: Strategy Pattern Example (3)

Code Implementation

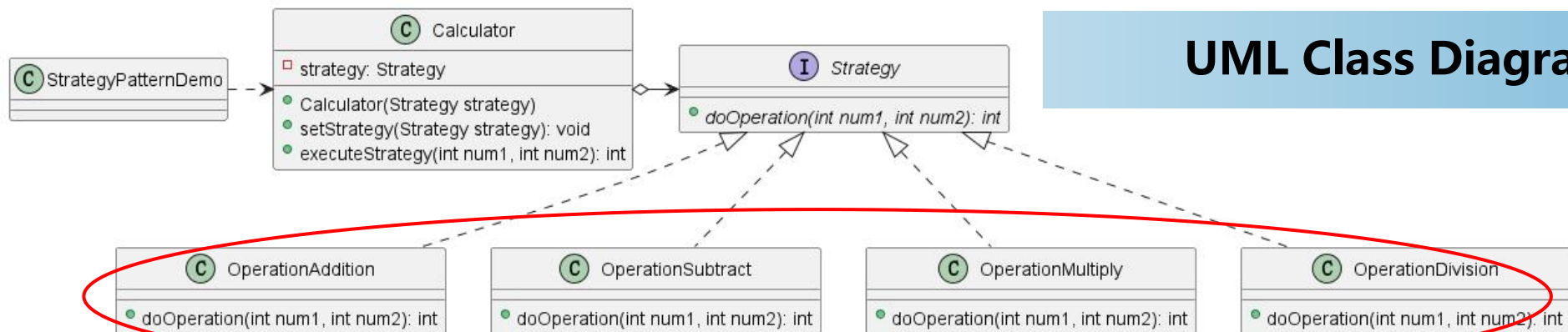
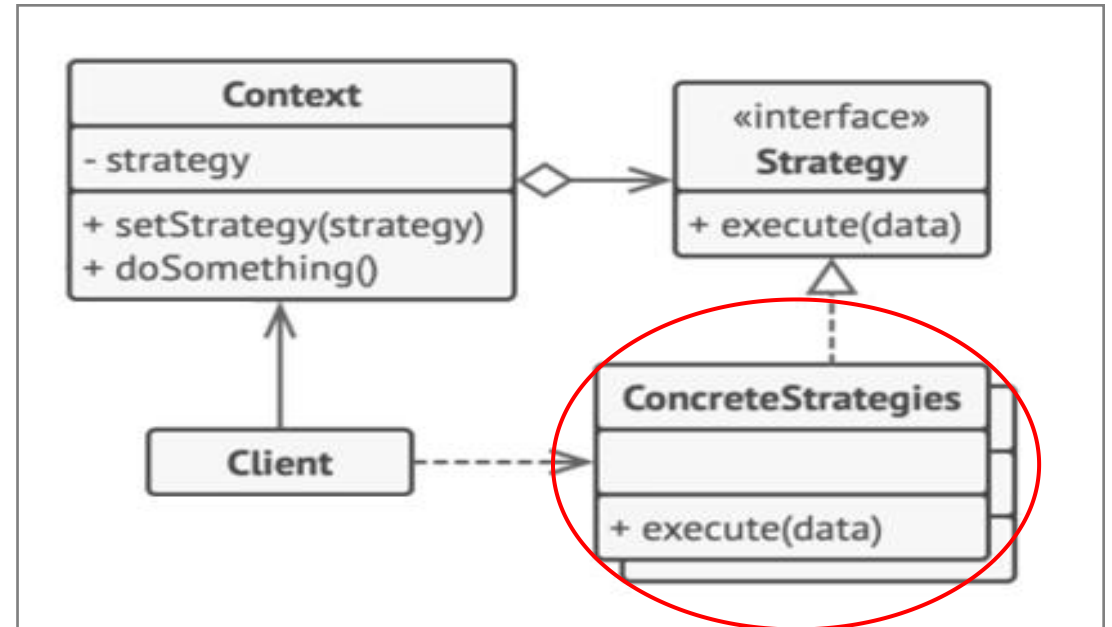
② Create four concrete strategy classes

```
public class OperationAddition implements Strategy {  
    @Override  
    public int doOperation(int num1, int num2) {  
        return num1 + num2;  
    }  
}
```

Addition

```
public class OperationSubtract implements Strategy {  
    @Override  
    public int doOperation(int num1, int num2) {  
        return num1 - num2;  
    }  
}
```

Subtract



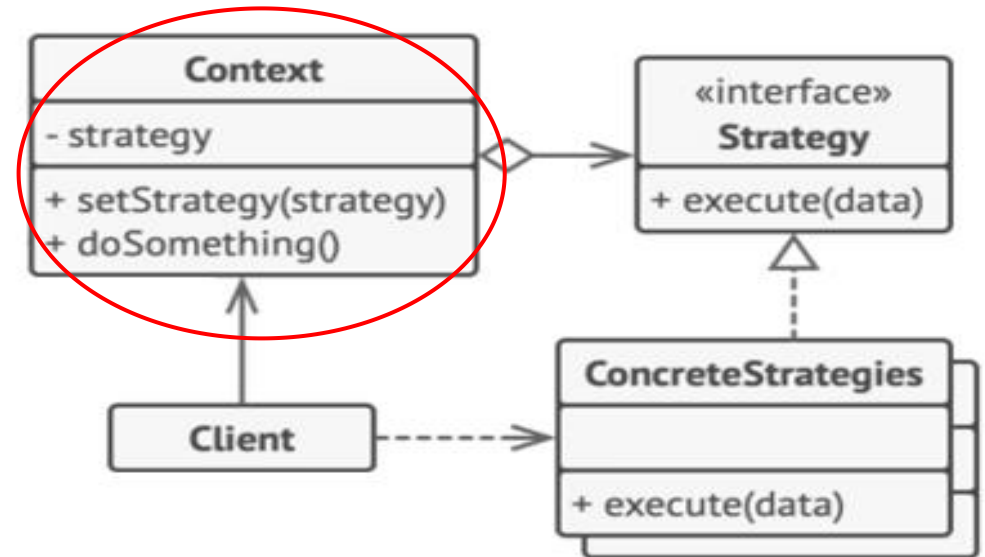
UML Class Diagram Design

Experiment Steps: Strategy Pattern Example (4)

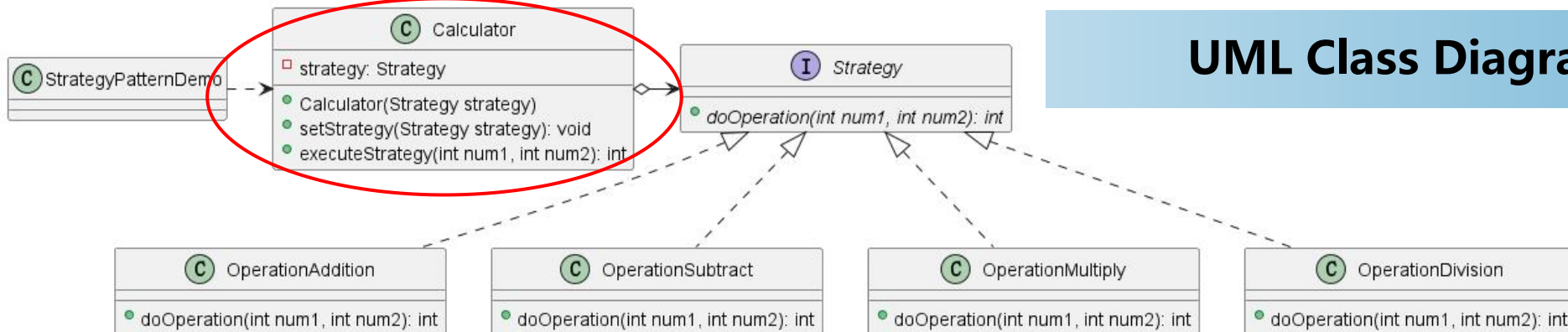
Code Implementation

③ Create Calculator class to act as Context role

```
public class Calculator {  
    private Strategy strategy;  
  
    public Calculator(Strategy strategy){  
        this.strategy = strategy;  
    }  
    public void setStrategy(Strategy strategy) {  
        this.strategy = strategy;  
    }  
    public int executeStrategy(int num1, int num2){  
        return strategy.doOperation(num1, num2);  
    }  
}
```



UML Class Diagram Design

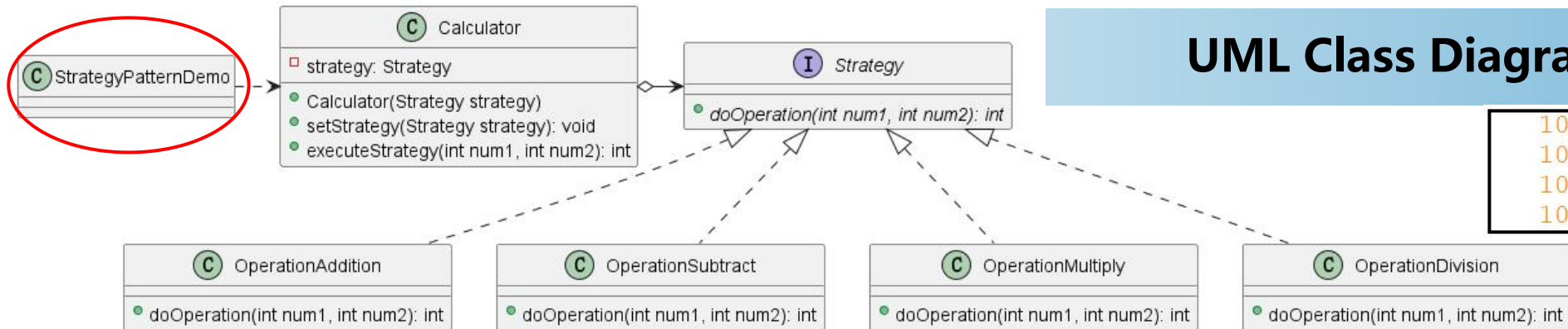
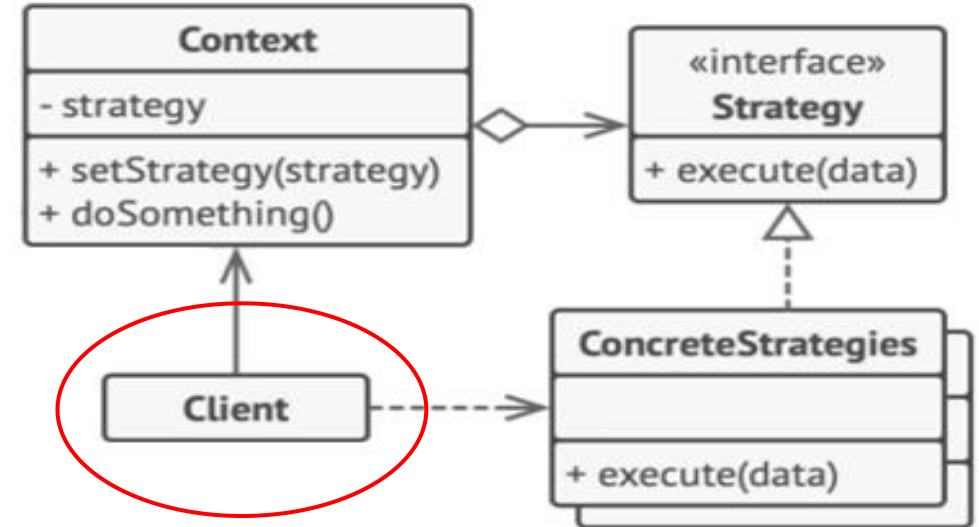


Experiment Steps: Strategy Pattern Example (5)

Code Implementation

④ Client uses Context to set different strategies

```
public class StrategyPatternDemo {  
    public static void main(String[] args) {  
  
        Calculator calc= new Calculator(new OperationAddition());  
        System.out.println("10 + 5 = " + calc.executeStrategy(10, 5));  
  
        calc.setStrategy(new OperationSubtract());  
        System.out.println("10 - 5 = " + calc.executeStrategy(10, 5));  
  
        calc.setStrategy(new OperationMultiply());  
        System.out.println("10 * 5 = " + calc.executeStrategy(10, 5));  
        calc.setStrategy(new OperationDivision());  
        System.out.println("10 / 5 = " + calc.executeStrategy(10, 5));  
    }  
}
```



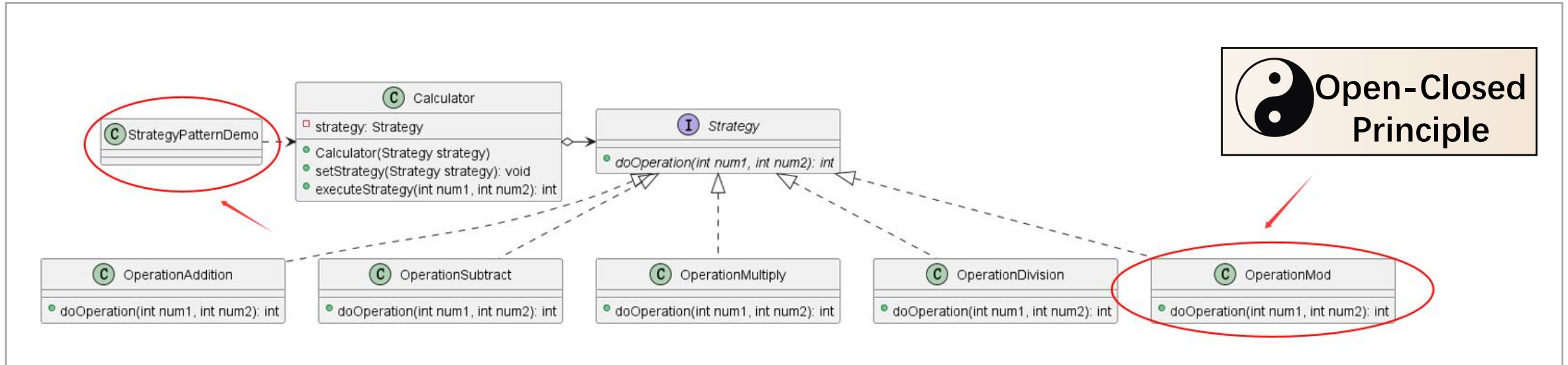
UML Class Diagram Design

10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2

Experiment Steps: Strategy Pattern Example (6)



Let's think: How to add a modulo operation?



```
public class StrategyPatternDemo {
    public static void main(String[] args) {
        Calculator calc= new Calculator(new OperationAddition());
        System.out.println("10 + 5 = " + calc.executeStrategy(10, 5));
        calc.setStrategy(new OperationDivision());
        System.out.println("10 / 5 = " + calc.executeStrategy(10, 5));

        //Switch to the modulo operation strategy
        calc.setStrategy(new OperationMod());
        System.out.println("10 % 5 = " + calc.executeStrategy(10, 5));
    }
}
```

10	+	5	=	15
10	-	5	=	5
10	*	5	=	50
10	/	5	=	2
10	%	5	=	0

Experiment Steps: Two Types of Firepower Power-ups

➤ Tasks to be completed:

- When the hero aircraft collects a **Firepower Supply**, its bullet trajectory immediately switches to a **spread pattern**—for example, from straight shot to firing 5 bullets at once in a fan-like formation. 
- When the hero aircraft collects a **Super Firepower Supply**, the trajectory changes to a **circular pattern**, firing 20 bullets simultaneously in all directions. 

Note: This lab does not involve multithreading. Once the trajectory changes, it won't revert back.



Assignment Submission

- **Submission requirements**

- ① Project code (the entire project folder)
- ② Experiment report (use report template)

- **Deadline**

Submit your work to the **HITsz Grader** assignment submission platform by the deadline: Tuesday of Week 14.

Login URL: <https://grader.cs-lab.top/>

The default username and password are your student ID.



Let's begin the experiment!

THANK YOU